6-1 · 실시간 데이터와 Kafka · 이론 1H

실시간 데이터의 특징 — 배치 vs 스트리밍

모아서 한꺼번에 vs 생기는 대로 흘려보내며

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 배치와 스트리밍의 차이를 처리 단위·지연 관점에서 설명한다

② 「늦게 알면 가치가 없어지는가」로 스트리밍 필요 여부를 판단한다

③ 이벤트 시간과 처리 시간이 왜 다른지 말할 수 있다

④ Kafka 가 보내는 쪽과 받는 쪽을 떼어 놓는 이유를 설명한다

다음 2.5H 실습에서 Kafka 를 직접 띄워 「이벤트가 쌓이고, 읽어도 사라지지 않는다」를 확인합니다

모아서 한꺼번에 vs 생기는 대로 흘려보내며

배치 (1~5주차)

데이터를 모아 두었다가 정해진 시각에 한꺼번에 처리

- 처리 단위: 기간(하루·한 시간)
- 지연: 분~시간
- 정해진 시각에 시작하고 끝난다
- 하루치 판매 정산이면 충분

VS

스트리밍 (6주차)

데이터가 생기는 즉시 흘려보내며 처리

- 처리 단위: 이벤트 하나하나
- 지연: 밀리초~초
- 시스템이 계속 떠 있다
- 결제 이상 탐지·실시간 재고

여섯 관점으로 나란히 — 실시간이 항상 좋은 것은 아니다

관점	배치 (1~5주차)	스트리밍 (6주차)
처리 단위	기간(하루·한 시간)	이벤트 하나하나
지연	분~시간	밀리초~초
실행	정해진 시각에 시작·종료	계속 떠 있음
재처리	다시 돌리면 됨	보존된 이벤트를 되감아 읽음
복잡도	단순	순서·중복·지연 도착을 다룸
비용	돌 때만	항상 떠 있음

스트리밍은 계속 떠 있어야 하고 다뤄야 할 문제가 늘어난다 — 판단 기준은 하나, 「늦게 알면 가치가 없어지는가」

상황 여섯 개 — 배치인가, 스트리밍인가

상황	배치/스트리밍	왜
월말 정산	배치	늦어도 정확하면 된다
결제 이상 탐지	스트리밍	승인 전에 막아야 의미가 있다
추천 모델 학습	배치	몇 시간 늦어도 된다
재고 실시간 반영	스트리밍	품절 상품을 팔면 안 된다
일별 리포트	배치	하루 단위면 충분
서비스 장애 알림	스트리밍	즉시 알아야 한다

일별 정산은 늦어도 정확하면 되고, 결제 이상 탐지는 승인 전에 막아야 의미가 있다

「언제 일어났나」와 「언제 도착했나」는 다르다

1. **이벤트 시간(event time)**
   그 일이 실제로 일어난 시각 — 주문이 접수된 시각

2. **처리 시간(processing time)**
   우리 시스템이 그것을 받은 시각

3. **둘이 벌어진다**
   네트워크 지연·모바일 오프라인으로 몇 초에서 몇 시간까지

스트리밍 설계의 핵심 결정

늦게 온 이벤트를
언제까지 기다릴까

> 「오후 3시대 매출」은 어느 시각을 쓰느냐로 답이 달라진다 — 배치는 하루 뒤 집계라 잘 안 보이던 문제. 이번 과정은 개념만, 워터마크 처리는 다루지 않음

4. Kafka 가 하는 일

보내는 쪽과 받는 쪽을 떼어 놓는 중간 저장소

보내는 쪽	Kafka	받는 쪽
주문 시스템은 Kafka 에만 보낸다	이벤트 스트림을 안전하게 받아 둔다	재고·알림·분석이 각자 구독

받는 쪽이 늘거나 죽어도 보내는 쪽은 그대로 — 각자 자기 속도로 읽는다

2-5 크롤러의 fetch·parse·load 처럼 한 스크립트에 묶였던 것을 그 사이에 들어가 떼어 놓는다

Kafka 가 없으면, 있으면

Kafka 가 없으면

주문 시스템이 재고·알림·분석에 각각 직접 보낸다

- 받는 쪽이 하나 늘면 보내는 쪽을 고친다
- 받는 쪽이 죽으면 데이터를 잃는다
- 받는 쪽이 느리면 보내는 쪽이 막힌다

VS

Kafka 가 있으면

주문 시스템은 Kafka 에만 보낸다

- 새 소비자가 구독만 하면 된다
- Kafka 에 남아 있어 나중에 읽는다
- 각자 자기 속도로 읽는다

정리

이 1시간에서 꼭 가져갈 것

1. 스트리밍은 「늦게 알면 가치가 없어지는」 일에 쓴다 — 항상 좋은 것이 아니다
2. 스트리밍은 계속 떠 있어야 하고 순서·중복·지연 도착을 다뤄야 한다
3. 이벤트 시간과 처리 시간은 다르고, 어느 것을 쓰느냐로 집계가 달라진다
4. Kafka 는 보내는 쪽과 받는 쪽을 떼어 놓는 중간 저장소다
5. 받는 쪽이 늘거나 죽어도 보내는 쪽은 그대로다

6-1 · 이론 1H 마무리

다음은 Kafka 핵심 구조와 첫 실행 (2.5H)

토픽·파티션·오프셋을 우체국 비유로 잡고, Docker 로 브로커를 띄워 CLI 로 메시지를 주고받습니다

- ✓ 브로커·토픽·파티션·오프셋 낱말 잡기
- ✓ apache/kafka:3.7.0 단일 브로커 기동
- ✓ hello-stream 토픽에 메시지 송수신
- ✓ 읽어도 사라지지 않는 것을 눈으로 확인

준비할 것

Docker 실행 · Total Memory 7.7GiB 안팎(설정 8GB)